

PROJECT ORION

Same Agent, Two Architectures

When MCP Wins (And When It Doesn't)

Hands-On Lab Guide

Community Summit NA 2026 | Nashville, TN | October 11-15, 2026

Presented by Mariano Gomez, CPTO at Mekorma

What You Will Build

Component	Technology	Purpose
Project Orion database	SQL Server	Stores sprint metrics, work items, health data
GitHub repository	GitHub CLI	Tracks issues correlated to SQL data
Connector agent	Copilot Studio	Demo Part 1: single source, connector wins
Custom MCP Server	TypeScript, Node.js	Exposes SQL data to Copilot Studio
MCP agent	Copilot Studio	Demo Part 2: cross-source reasoning
Project health dashboard	Next.js, React	Visualizes SQL data before the agent demo

"When your agent needs to reason across sources it did not know about at design time, that is your MCP signal."

Prerequisites

Before starting, confirm you have the following installed and configured:

VS Code	Version 1.82 or later
Node.js	Version 18 or later with npm
TypeScript	Installed globally: npm install -g typescript
GitHub CLI	winget install --id GitHub.cli (Windows)
Git + Git Bash	Git Bash required on Windows for shell scripts
SQL Server	SQL Server Express (free) running locally
SSMS or Azure Data Studio	For running the setup SQL script
Azure Dev Tunnels CLI	winget install Microsoft.DevTunnel (Windows)
GitHub account	Personal account is sufficient
Copilot Studio license	Trial available at copilotstudio.microsoft.com
GitHub Personal Access Token	repo scope required

Repo Structure

After cloning the repo, you will find the following structure:

project-orion-session/	
README.md	# This file
docs/	
lab-guide.pdf	# PDF version of this guide
sql/	
project-orion-setup.sql	# Database schema and seed data
github/	
create-github-issues.sh	# Creates all 35 GitHub issues
mcp-server/	
copilot-mcp-scaffold-prompt.md	# Prompt: complete MCP Server scaffold
dashboard/	
copilot-nextjs-dashboard-prompt.md	# Prompt: scaffold the dashboard
copilot-studio/	
copilot-studio-connector-agent-prompt.md	# Prompt: connector agent (Demo Part 1)
copilot-studio-agent-prompt.md	# Prompt: MCP agent (Demo Part 2)

**Step
1****Set Up the SQL Server Database**

The Project Orion database is the foundation of everything. Set this up first.

1.1 Run the setup script

Open SQL Server Management Studio or Azure Data Studio and execute:

```
sql/project-orion-setup.sql
```

The script creates the ProjectOrion database, three tables (Sprints, WorkItems, HealthMetrics), seeds 5 sprints of history, 70 work items, and 30 days of health metrics, then runs verification queries.

1.2 Verify the data

Sprint	Status	Planned	Completed	Rollover
Sprint 1	Completed	42	40	2
Sprint 2	Completed	44	43	1
Sprint 3	Completed	45	38	7
Sprint 4	Completed	44	33	11
Sprint 5	Active	46	14	0

Latest health metric should show: ReleaseReadinessScore: 52 | BugCriticalCount: 3 | BlockerCount: 3 | VelocityTrend: Declining

1.3 Connection string reference

Local SQL Server (demo):

```
Server=localhost;Database=ProjectOrion;Trusted_Connection=True;TrustServerCertificate=True;
```

Azure SQL (production swap):

```
Server=<server>.database.windows.net;Database=ProjectOrion;User Id=<user>;Password=<pass>;Encrypt=True;
```

**Step
2****Create the GitHub Repository and Issues**

2.1 Authenticate with GitHub CLI

```
gh auth login
```

Select: GitHub.com, HTTPS, Yes, Login with a web browser. Complete the browser flow.

2.2 Update the script

Open `github/create-github-issues.sh` and update line 22:

```
REPO="your-github-username/project-orion"
```

2.3 Run the script

```
chmod +x github/create-github-issues.sh
./github/create-github-issues.sh
```

The script creates the repository, labels, all 35 issues, and prints a summary with the SQL correlation map.

2.4 SQL to GitHub correlation map

SQL Record	GitHub Issue
Critical blocker: auth token refresh	#25
Critical blocker: dashboard regression	#26
Critical blocker: data sync drops records	#27
Stalled feature: notification preferences UI	#28
Performance theme across Sprint 3-5	#13, #18, #26, #29, #31

Step 3

Build the Custom MCP Server

3.1 Create and open the project folder

Create a new folder called `project-orion-mcp-server` and open it in VS Code.

3.2 Scaffold with GitHub Copilot

Open Copilot Chat (Ctrl+Alt+I), paste the contents of:

```
mcp-server/copilot-mcp-scaffold-prompt.md
```

This single prompt scaffolds the complete project: all seven MCP tools, HTTP transport with `StreamableHTTPServerTransport` in stateless mode, SQL and Entra ID auth options, CORS, environment config, and a README. No separate HTTP transport step required.

3.3 Install, configure, and build

```
npm install\nnpm run build
```

Copy `.env.example` to `.env`. For local demo use SQL Authentication:

```
DB_SERVER=localhost\nDB_DATABASE=ProjectOrion\nDB_TRUSTED_CONNECTION=false\nDB_USER=sa\nDB_PASSWORD=your-sql-password\nDB_TRUST_SERVER_CERTIFICATE=true\nMCP_HTTP_PORT=3000\nMCP_TRANSPORT=http
```

For Azure SQL production, set `DB_AUTH_TYPE=azure-active-directory-default` to use Entra ID.

3.4 Start and verify

```
npm run start:http
```

Before connecting Copilot Studio, verify the server is reachable:

```
curl http://localhost:3000/health
```

Expected: {"status":"ok","server":"project-orion-mcp-server"}

3.5 The seven MCP tools

Tool	Description
get_current_sprint	Active sprint details and completion rate
get_sprint_history	Velocity trend across all completed sprints
get_critical_work_items	All open Critical and High priority blockers
get_work_items_by_sprint	Work items filtered by sprint and status
get_latest_health_metrics	Most recent health snapshot and readiness score
get_health_metrics_trend	Release readiness score trend over N days
get_stalled_work_items	Items with no recent activity

Step
4

Expose the MCP Server via Dev Tunnels

Copilot Studio is a cloud service and cannot reach your localhost directly. Azure Dev Tunnels creates a secure bridge between your local MCP Server and Copilot Studio.

4.1 Install and authenticate

```
winget install Microsoft.DevTunnel
```

Restart your terminal, then authenticate:

```
devtunnel user login
```

Sign in with the same Microsoft account you use for Copilot Studio.

4.2 Start the tunnel

With the MCP Server running in HTTP mode, open a separate terminal and run:

```
devtunnel host -p 3000 --allow-anonymous
```

You will receive a tunnel URL. Your Copilot Studio SSE endpoint is:

```
https://xxxxxxx-3000.devtunnels.ms/sse
```

Keep this terminal open while using Copilot Studio. The tunnel closes when the terminal closes. Note the full URL including /sse at the end.

This agent is intentionally limited to GitHub Issues only. The limitation IS the demo.

5.1 Create the agent

Go to copilotstudio.microsoft.com, click Create, and paste:

```
copilot-studio/copilot-studio-connector-agent-prompt.md
```

5.2 Add the GitHub connector

In your agent: **Tools > Add a tool > Connector**. Search for GitHub and connect with your account.

5.3 Test prompts

Prompt	Expected result
What critical bugs are open in Project Orion?	Returns #25, #26, #27 with titles. Connector wins.
What is the overall health of Project Orion right now?	Incomplete. Agent can only see GitHub. Cannot show release readiness or sprint metrics. This is the ceiling.

Transition line after the ceiling moment: "The question changed. Now let's change the architecture."

6.1 Create a NEW agent

Create a separate agent (do not modify the connector agent). Paste:

```
copilot-studio/copilot-studio-agent-prompt.md
```

6.2 Add the custom MCP Server

In your agent: **Tools > Add a tool > Model Context Protocol**. Enter your Dev Tunnel SSE endpoint. Select No authentication. Enable all seven tools.

6.3 Create a GitHub OAuth App

GitHub MCP requires a GitHub OAuth App for authentication. Complete this before configuring Copilot Studio.

- Go to github.com/settings/developers
- Click **OAuth Apps** then **New OAuth App**
- Application name: Project Orion MCP. Homepage URL: <http://localhost:3000>. **Authorization callback URL:** This field is required. Enter a placeholder:
<https://global.consent.azure-apim.net/redirect/github-mcp> (you will replace this with the

real URL Copilot Studio generates in the next step.)

- Click **Register application**
- Click **Generate a new client secret**
- Copy and save both the **Client ID** and **Client Secret** before leaving the page

6.4 Add the GitHub MCP Server in Copilot Studio

In your agent: **Tools > Add a tool > Model Context Protocol**

Important: GitHub does not publish an OIDC discovery document. The Dynamic Discovery option will fail with an "Invalid URI" error. Select **Manual** configuration instead.

In the MCP tool dialog:

- Select **Manual** (not Dynamic Discovery)
- Enter the GitHub MCP endpoint URL: `https://api.githubcopilot.com/mcp/`
- Select **OAuth** as the authentication type
- Copy the **Redirect URL** shown by Copilot Studio. It will look like:
`https://global.consent.azure-apim.net/redirect/`
- Go back to your GitHub OAuth App, **replace the placeholder Authorization callback URL** with the exact Redirect URL from Copilot Studio, then save
- Enter the OAuth values below and save
- When prompted, authorize the OAuth app with your GitHub account

Setting	Value
Authorization URL	<code>https://github.com/login/oauth/authorize</code>
Token URL	<code>https://github.com/login/oauth/access_token</code>
Refresh URL	<code>https://github.com/login/oauth/access_token</code>
Scope	<code>repo read:org</code>
Client ID	From your GitHub OAuth App
Client Secret	From your GitHub OAuth App

6.5 Test prompts

#	Prompt	Validates
1	What is the current release readiness score?	SQL only. Score 52, declining.
2	What critical bugs are currently open?	GitHub only. Issues #25, #26, #27.
3	What is blocking the release right now?	Cross-source. Both GitHub issues AND SQL metrics in one answer.
4	Any patterns between GitHub issues and sprint health?	Pattern detection. Performance theme + velocity decline.

5	Which work items have been stalled the longest?	Stalled work. Notification feature, #28, 12+ days.
---	---	--

Prompt 3 is the money shot. If the agent references GitHub issues #25, #26, #27 AND SQL BlockerCount=3 and ReleaseReadinessScore=52 in a single coherent answer, the cross-source reasoning is working end to end.

Step
7

Build the Next.js Dashboard

The dashboard visualizes the SQL data for the audience before the agent demo. It sets the narrative context.

7.1 Create and scaffold

Create a folder called `project-orion-dashboard`, open it in VS Code, then paste:

```
dashboard/copilot-nextjs-dashboard-prompt.md
```

7.2 Configure and run

Copy `.env.example` to `.env.local` with your SQL Server settings, then:

```
npm install
```

```
npm run dev
```

Open `http://localhost:3001` (or the port shown in the terminal).

7.3 Verify the four panels

Panel	Expected value
Sprint Health	Sprint 5, 39.13% completion, Declining
Bug Tracker	7 open bugs, 3 critical, #25 #26 #27 listed
Release Readiness	52/100, yellow gauge, 'Release at risk'
Team Velocity	Peak Sprint 2 (43pts), decline to Sprint 4 (33pts)

The header status pill should show **"At Risk"** in yellow.

Decision Framework

Use this to evaluate your own projects after the lab:

Source Complexity	Reasoning Required	Recommendation
Single, static	Retrieve	Connector wins, every time
Single, static	Reason	Connector with a well-designed prompt
Multiple, dynamic	Retrieve	Consider MCP, evaluate maintenance cost
Multiple, dynamic	Reason	MCP is the right architectural call

The honest warning: The most expensive architectural mistake is reaching for MCP because it sounds modern. Complexity has a cost. Make sure your problem justifies it.

Run It Yourself: Cheat Sheet

Setup checklist (before going on stage)

- SQL Server running locally
- MCP Server running: `npm run start:http`
- Dev Tunnel active: `devtunnel host -p 3000 --allow-anonymous`
- Next.js dashboard running: `npm run dev`
- Both Copilot Studio agents open in separate browser tabs
- GitHub issues open in a third browser tab

Demo Part 1: Connector Agent

- Show the Next.js dashboard briefly
- Switch to the connector agent
- Run: "What critical bugs are currently open in Project Orion?" (connector wins)
- Run: "What is the overall health of Project Orion right now?" (connector hits ceiling)
- Transition: "The question changed. Now let's change the architecture."

Demo Part 2: MCP Agent

- Show the architecture diagram slide
- Open VS Code, show the custom MCP Server tool definitions (30 seconds)
- Switch to the MCP agent
- Run: "What is blocking the Project Orion release right now?" (cross-source money shot)

- Run: "Are there any patterns between our open GitHub issues and sprint health metrics?"
- Decision anchor: "When your agent needed to reason across sources it did not know about at design time. That is your MCP signal."

Resources

MCP Official Documentation: modelcontextprotocol.io

Copilot Studio Documentation: learn.microsoft.com/copilot-studio

GitHub MCP Server: github.com/github/github-mcp-server

Azure Dev Tunnels: learn.microsoft.com/azure/developer/dev-tunnels

Community Summit NA 2026 | Nashville, TN | October 11-15, 2026

Mariano Gomez, CPTO at Mekorma